CAPSTONE DEFENSE FIELD GUIDE

AlvanzStock

System Navigation, Code Changes & Debugging

A hands-on study guide for understanding the application, explaining its design, and confidently making a small change while an instructor watches.

Django 6.1 · Python · SQLite · Tailwind CSS · Vanilla JavaScript
Prepared from the current project source · October 2026

How to use this guide

This is a map and a practice manual, not a line-by-line copy of the source. Learn the request flow and the few ownership rules first. When asked to change something, use the checklists and trace the behavior from the browser to the responsible view, model, and template.

> **Defense habit:** say what behavior you intend to change, identify the owning file, make one focused edit, then prove it with a test or a reproducible browser check. Do not start editing until you can describe the expected result.

Contents

1. Start and run the system
2. How a request travels
3. Pages and access by role
4. Data model and stock rules
5. Important workflows
6. How to change the code
7. Debugging playbook
8. Testing and verification
9. Defense rehearsal and questions
10. Caveats and production safety

1. Start and run the system

Run commands from the project root, the folder containing `manage.py`. This repository has a local virtual environment named `venv`.

```
# Install/update Python dependencies
./venv/bin/pip install -r requirements.txt

# Apply database migrations
./venv/bin/python manage.py migrate

# Optional demo users and sample products/sales
./venv/bin/python manage.py seed_data

# Install frontend build dependencies once, then compile CSS
npm install
npm run build:css

# Start the local site
./venv/bin/python manage.py runserver 127.0.0.1:8000
```

Open `http://127.0.0.1:8000/`. Stop the development server with `Ctrl+C`. If you change template classes, rebuild Tailwind with `npm run build:css`; the browser uses `static/css/output.css`, not the Tailwind source config directly.

The optional seed command creates demo accounts if absent: `admin / admin123`, `cashier / cashier123`, and `staff / staff123`. Those credentials are for local demonstrations only. Seeded categories prevent the command from reseeding demo inventory/sales on later runs.

> **Do not confuse environments:** `db.sqlite3` is the development database. Django tests create a separate temporary test database and destroy it afterward. A successful test run does not create or delete your demo data.

2. How a request travels

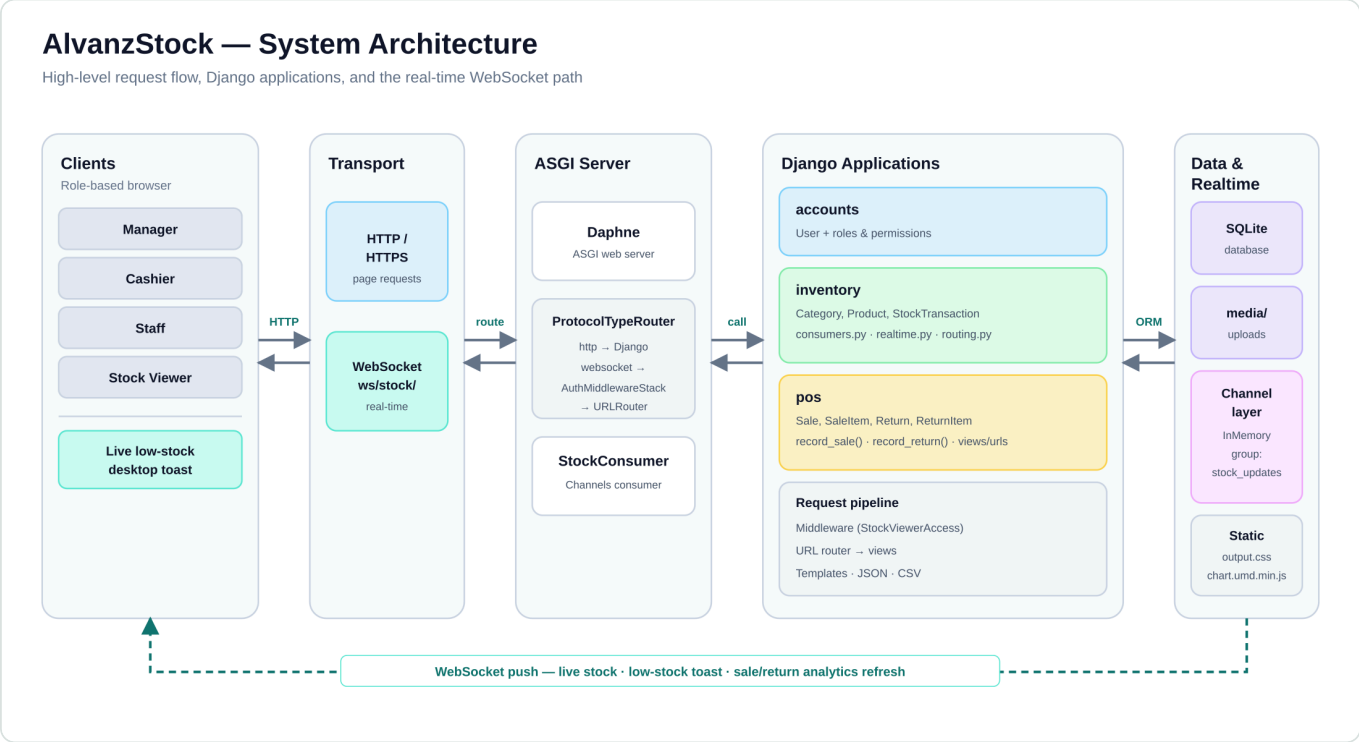


Figure 1. High-level system architecture: clients, transport (HTTP + WebSocket), ASGI server, Django applications, and data/realtime stores.

Browser URL → root URL configuration → app URL → view → model/query → template or JSON → browser

- `alvanzstock/urls.py` is the top-level router. It includes account, inventory, and POS URLs.
- `inventory/urls.py` and `pos/urls.py` map paths to functions in their app's `views.py`.
- A view checks access, reads input, asks models for data or saves changes, then renders a template or returns JSON.
- Templates in `templates/` create the HTML. Most extend `templates/base.html`, which provides the shared layout and navigation.
- POS interactions use ordinary HTML plus JavaScript in `templates/pos/pos_screen.html`. Checkout posts JSON to a Django view.
- Models define database fields and important behavior. Migrations translate model changes into database changes.

Example: opening `/stock/new/` matches `inventory/urls.py`, invokes `stock_transaction_create`, validates `StockTransactionForm`, saves the transaction and user, then renders `templates/inventory/stock_form.html`. A valid submission redirects to the affected product detail page.

3. Pages and access by role

Page or action	Route	Owner	Access
Dashboard	`/`	`inventory/views.py : dashboard`	Signed-in users; Stock Viewer is redirected to Live Stock.
Products (read)	`/products/`	`inventory/views.py : product_list`	Signed-in users (read-only for Cashier and Staff).

Page or action	Route	Owner	Access
Add / edit / delete product	`/products/new/`, `/products/<id>/edit/`, `/products/<id>/delete/`	`inventory/views.py`, `ProductForm`	Manager role; protected by `inventory_manager_required`.
Categories (read)	`/categories/`	`inventory/views.py`	Signed-in users (read-only for Cashier and Staff).
Add / delete category	`/categories/new/`, `/categories/<id>/delete/`	`inventory/views.py`	Manager role; protected by `inventory_manager_required`.
Stock movement history (read)	`/stock/`	`inventory/views.py`	Signed-in users (read-only for Cashier and Staff).
Record / delete stock movement (restock)	`/stock/new/`, `/stock/<id>/delete/`	`inventory/views.py`	Manager role; protected by `inventory_manager_required`. Cashier/Staff are redirected to `/pos/`.
Low-stock alerts	`/low-stock/`	`inventory/views.py: low_stock`	Signed-in users (restock buttons hidden for Cashier and Staff).
Live stock and WebSocket feed	`/real-time-stock/`, `ws/stock/`	`inventory/views.py`, `inventory/consumers.py`	Signed-in users; Stock Viewer is allowed.
POS and checkout	`/pos/`, `/pos/checkout/`	`pos/views.py`	Signed-in users except Stock Viewer.
Receipt	`/receipt/<id>/`	`pos/views.py: sale_receipt`	Signed-in users except Stock Viewer.
Sales history and reports	`/sales/`, `/reports/sales/`, `/reports/inventory/`	`pos/views.py`	Manager role.
Analytics dashboard	`/reports/analytics/`, `/reports/analytics/data/`, `/reports/analytics/export/`	`pos/views.py`	Manager role.
Django admin	`/admin/`	`accounts/admin.py`, app admin files	Requires Django admin staff/superuser permission.

Roles and where restrictions live

- **Manager:** `User.is_manager` is true; full access, including inventory writes and sales/report views protected by `manager_required`.
- **Cashier:** full POS (checkout, receipts) and read-only inventory; inventory write actions are blocked by `inventory_manager_required` and redirected to `/pos/`.
- **Staff:** read-only inventory and read-only views; inventory write actions are blocked by `inventory_manager_required` and redirected to `/pos/`.
- **Stock Viewer:** sees only Live Stock and logout in navigation. `accounts/middleware.py` also redirects direct requests to other view names, so hiding links is not the only security control.

Role guards are shared from `accounts/decorators.py`: `manager_required` (redirects to the dashboard) protects sales/report views, and `inventory_manager_required` (redirects to `/pos/`) protects inventory writes. Direct URL access is always blocked, not just hidden buttons.

Create users at `/admin/`. Choosing the Stock Viewer role does not automatically make a user an admin. Admin access is separately controlled by `is_staff` and permissions.

4. Data model and stock rules

Figure 2 shows the schema; the table lists each model and its key fields. Foreign keys (FK) implement the relationships, and most are nullable so history survives when a category, product, or user is removed (`SET_NULL`) or cascades with its parent (`CASCADE`).

Model	Purpose	Important fields / relationship
`User`	Authentication and role.	`role` ; custom model in `accounts/models.py` .
`Category`	Groups products.	`name` ; a product may have no category.
`Product`	Current catalog and stock count.	`sku` , `image` , prices, `quantity` , `min_stock_level` , `is_active` .
`StockTransaction`	Records a stock movement.	Product, type, quantity, reference, user, timestamp.
`Sale`	One checkout/receipt.	Cashier, subtotal, discount, total, tender, change, receipt number.
`SaleItem`	Products and quantities within a sale.	Sale, product, quantity, unit price, line subtotal. Tracks `returned_quantity` / `returnable_quantity` .
`Return`	One refund against a sale.	Sale, processed-by user, refund amount and method, reason, receipt number.
`ReturnItem`	Products and quantities returned within a return.	Return, sale item, product, quantity, unit price, line subtotal, `restock` flag.

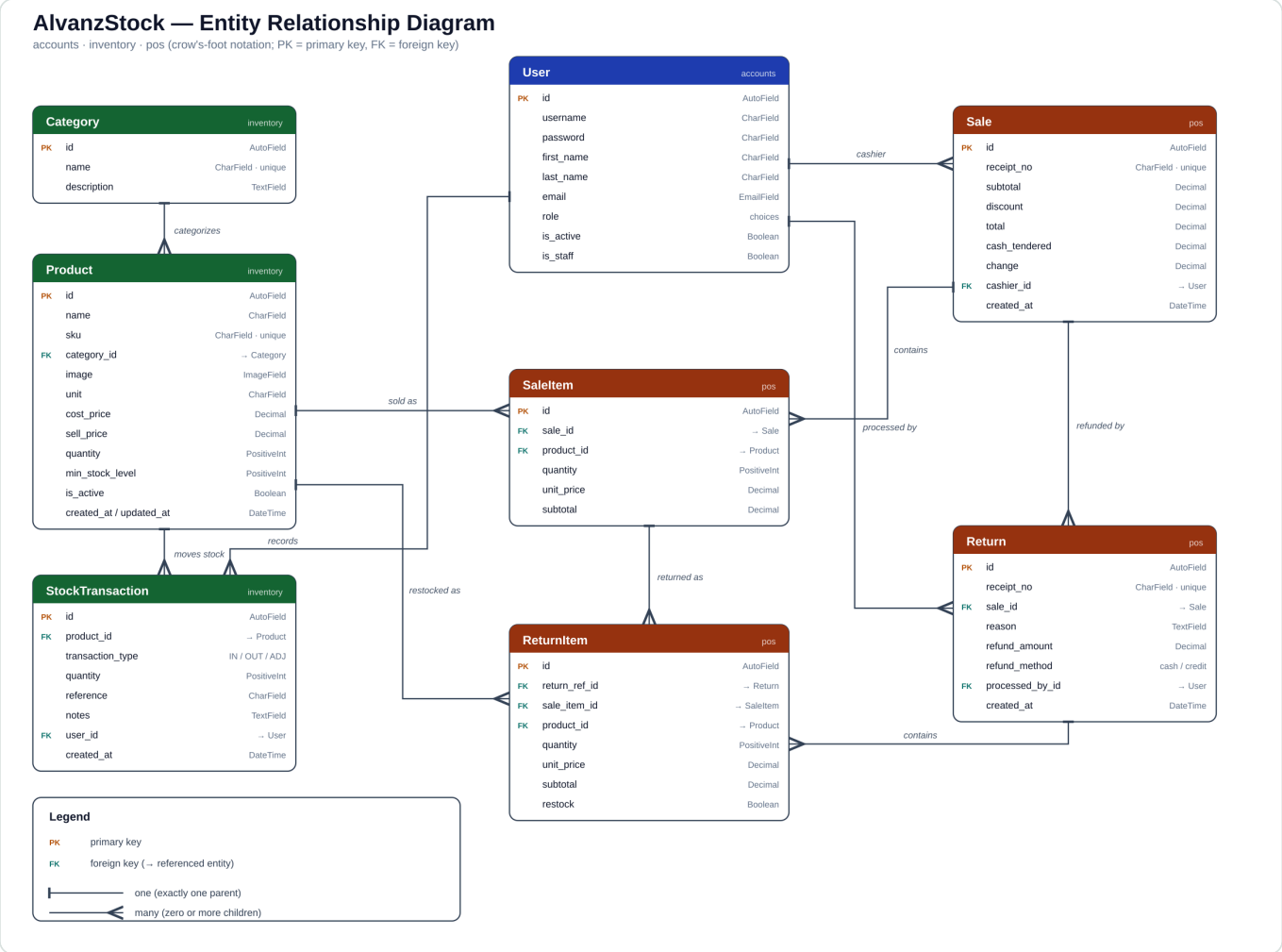


Figure 2. Entity relationship diagram across the accounts, inventory, and pos apps (crow's-foot notation).

The stock invariant

`Product.quantity` is the current count displayed to users. A new `StockTransaction` updates it in `StockTransaction.save()`; deleting a transaction calls `reverse_stock()`. A POS sale creates stock-out transactions, so the same stock mechanism updates inventory from checkout.

- Stock In adds the movement quantity.
- Stock Out subtracts it and clamps at zero in the model.
- `is_low_stock` means `quantity <= min_stock_level`.
- The current implementation treats every non-IN movement, including ADJUST, as subtracting stock. If asked for positive and negative adjustments, clarify the rules and update both apply and reverse behavior plus tests.

> **Audit point:** the product admin currently allows editing `quantity` directly. Direct edits do not create a StockTransaction, so they can make the count and movement history disagree. A more auditable design would make quantity read-only in admin and require a transaction/adjustment.

5. Important workflows

Product images

1. `Product.image` is an optional Django `ImageField` saved under `media/products/`. Pillow validates image files.
2. `ProductForm` includes the image field.

3. `templates/inventory/product_form.html` uses multipart encoding; create/edit views pass `request.FILES`.
4. In development, `MEDIA_ROOT` and `MEDIA_URL` are configured in settings and URL serving is enabled only when `DEBUG` is true.
5. The POS product grid displays the image when present; otherwise it displays a placeholder. The image URL is passed in a data attribute for the cart.

Record stock movement

Form input → StockTransactionForm validation → assign request.user → save transaction → update Product.quantity → redirect to product detail

Transaction deletion is a POST-only action. The model reverses the stock effect before deleting the history record. Test both creation and deletion effects when changing this logic.

POS checkout

Product cards → JavaScript cart → POST JSON → pos_checkout validation → record_sale atomic transaction → Sale + SaleItems + stock-out transactions → receipt

The browser computes totals for usability, but `pos_checkout` checks quantities, discount, tendered cash, and the cash multiplier again on the server. The model helper `record_sale()` wraps sale, items, and stock movements in a database transaction; if part fails, the database transaction can roll back.

Live stock

The page opens a WebSocket to `ws/stock/` and receives a full snapshot on connect, then per-product updates as stock changes. Updates are pushed by the server (event-driven), not polled. The consumer is `StockConsumer` in `inventory/consumers.py`; broadcasts are sent from `inventory/realtime.py` and triggered inside `StockTransaction.apply_to_stock()` / `reverse_stock()`, so every movement, sale, and return updates all connected clients at once. The JSON route `/real-time-stock/data/` remains as an HTTP fallback (used on load and while a socket reconnects).

Sales reporting

The manager-only sales report defaults to one day (today), accepts `?days=N` for 1–365 days, and includes both boundary dates. Print Report invokes the browser print dialog; print CSS hides controls and app navigation. The sales list also supports date filters.

Analytics dashboard

Date range → build_analytics() aggregates sales, refunds, and COGS → KPI cards + Chart.js charts → CSV export or browser Print-to-PDF

The manager-only page at `/reports/analytics/` fetches numbers from the JSON endpoint `/reports/analytics/data/` (which reuses `build_analytics(days)`), then renders KPI cards and three charts with the vendored Chart.js. Net sales = gross totals − refunds; gross profit = net sales − COGS (unit cost × quantity); margin is profit ÷ net sales. When a sale or return commits, `record_sale()` / `record_return()` call `broadcast_event("sale"/"return")`; `base.html` receives it and dispatches an `alvanz:sale_change` window event that refreshes the charts live. `analytics_export` streams the same figures as CSV with `csv.writer`.

6. How to change the code

Use this change checklist

1. Restate the requested behavior as an observable result. Ask about edge cases before guessing.
2. Find the owning route and view. Search the URL name or visible text, then follow the URL → view → template path.
3. Decide which layer owns the change: template/layout, form validation, view/query, model/data, role enforcement, or JavaScript interaction.
4. Make the smallest focused edit. If a model field changes, generate and review a migration.
5. Run the narrow test first. Then run relevant checks and manually verify the browser workflow.
6. Explain the files changed and evidence that the behavior works.

Example A: add search by SKU in POS

1. In `templates/pos/pos_screen.html`, add `data-sku="{{ p.sku }}"` to each product button.
2. In the search input handler, match the lowercase query against both `btn.dataset.name` and `btn.dataset.sku`.
3. Verify exact and partial SKU matching, mixed case, and a query with no results.
4. Keep checkout validation in `pos/views.py`; browser search must not become a security or stock validation control.

Example B: add a new page

1. Add a view to the owning app, usually protected with `@login_required`.
2. Add a named path in that app's `urls.py`.
3. Create a template under `templates/`, usually extending `base.html` and defining `title` and `content`.
4. Add desktop/mobile navigation links in `templates/base.html` if users should discover it.
5. If the Stock Viewer must not access it, review `allowed_view_names` in `accounts/middleware.py`; the middleware currently redirects every view not explicitly allowed.

Example C: add or change a database field

1. Change the model in the app's `models.py`.
2. Expose it through the correct ModelForm and template. For a file upload, include multipart encoding and pass `request.FILES`.
3. Run `venv/bin/python manage.py makemigrations` and inspect the generated migration.
4. Run `venv/bin/python manage.py migrate`.
5. Write a test for valid input, invalid input, and the rendered/useful output.

Example D: change low-stock behavior

Current condition: `quantity <= min_stock_level`. First clarify whether equality should trigger the alert. Update the central product property and any database query that duplicates this rule (dashboard and low-stock view), then test one product below, exactly at, and above its minimum. Centralizing the rule reduces inconsistent badges and reports.

7. Debugging playbook

Trace it from the symptom

1. **Reproduce:** write down the URL, role, inputs, actual result, expected result, and exact error text.
2. **Find the route:** inspect `alvanzstock/urls.py`, then the app URL file.

3. **Follow the data:** view → form/query/model → template or JSON → browser JavaScript.
4. **Use both consoles:** Django traceback in the terminal; browser Console and Network tabs for JavaScript, redirects, and missing assets.
5. **Check one layer at a time:** Is the route matched? Is the view reached? Does the query return data? Does the HTML contain it? Does the browser render it?
6. **Make a minimal correction, rerun the narrow test, then regression tests.**

Symptom	Check first
404 after adding a page	Is the app URL included at the project level? Is the route spelled and named correctly?
TemplateDoesNotExist	Does the template path passed to `render()` match its location under `templates/`?
New Tailwind class has no effect	Run `npm run build:css`, confirm the template is scanned, then hard-refresh the browser.
Image missing after submit	Check the form has `enctype="multipart/form-data"`, the view passes `request.FILES`, validation errors, and that the image file exists under MEDIA_ROOT.
Image URL is 404 locally	Check MEDIA_URL/MEDIA_ROOT and the DEBUG-only media route in `alvanzstock/urls.py`. Production needs separate media hosting.
Stock Viewer is redirected	Check the user's role and `allowed_view_names` in `accounts/middleware.py`. Being absent from the menu is not the same as being allowed.
Stock number is wrong	Inspect the transaction type, quantity, save/delete path, and whether someone edited Product.quantity directly in admin.
Checkout rejects the cart	Read the JSON response in Network tools and check current database stock, discount ≤ subtotal, cash ≥ total, and configured cash multiplier.
"No changes detected" after model edit	Confirm the model was saved and belongs to an installed app. Review existing migrations before trying to force a schema change.
New import fails locally	Install dependencies into the project venv with `venv/bin/pip install -r requirements.txt`; use the same interpreter to run Django.

Example: image upload says success but POS shows a placeholder

1. Open the product edit page. Does it show the saved-image preview?
2. Inspect the rendered POS product button. Does its `data-image` value contain a URL?
3. Open that URL directly. A 404 points to media storage/serving; no URL points to form/view/model persistence; a valid URL but missing cart thumbnail points to JavaScript data/rendering.

8. Testing and verification

```
# All tests
venv/bin/python manage.py test

# Focused examples
venv/bin/python manage.py test inventory.tests.ProductImageUploadTests
venv/bin/python manage.py test accounts.tests.StockViewerAccessTests
venv/bin/python manage.py test pos.tests.SalesReportTests

# Django settings and URL/template checks
venv/bin/python manage.py check

# Verify model/migration agreement
venv/bin/python manage.py makemigrations --check --dry-run

# Apply migrations
venv/bin/python manage.py migrate
```

Tests use Django's test client to request routes and inspect status, redirects, page content, context, and stored objects. Image tests can construct a small in-memory PNG with Pillow and a `SimpleUploadedFile`. Good tests verify outcomes, not merely that a view returns 200.

For manual browser verification, use one action per test: upload an image, view the product, open POS, add it to the cart, verify its thumbnail, then checkout with valid and invalid values. Use a Manager and Stock Viewer account separately when verifying permissions.

9. Defense rehearsal and likely questions

A short live demonstration

1. Log in as a Manager and show product inventory and the stock threshold.
2. Edit a product and upload a photo; show its preview and POS card.
3. Open Record Stock Movement, record a stock-in, and show the quantity change.
4. Open Live Stock and explain the WebSocket push (snapshot on connect, then per-product updates broadcast by the server).
5. Open POS, add the pictured item, show the cart thumbnail, and explain browser-side totals versus server-side validation.
6. Complete a sale, print the receipt, and point out the Sale, SaleItem, and stock-out records.
7. Open the sales report, choose a day range, print it, then show the Stock Viewer's restricted navigation and route behavior.
8. Open Analytics, change the day range, show net sales / profit / margin KPIs and the charts, then complete a sale in another tab and point out the live refresh.
9. Finish with one test command and its result.

Questions to practice answering

Question	Answer outline
How does checkout update inventory?	The server validates the submitted cart; `record_sale()` atomically creates the sale and items and records stock-out transactions. The transaction model applies those movements to Product.quantity.

Question	Answer outline
Why validate on the server if JavaScript already checks?	Browser state can be modified or stale. Server checks are authoritative for inventory, discount, and payment.
What makes the stock page “real time”?	A Django Channels WebSocket (`ws/stock/`) pushes updates. The consumer sends a snapshot on connect and the server broadcasts each product change; the HTTP JSON route is only a fallback. This is push, not polling.
How do permissions work?	Authentication decorators require sign-in, manager views use `manager_required`, inventory writes use `inventory_manager_required` (Cashier/Staff are redirected to `/pos/`), and StockViewerAccessMiddleware allows only the stock page/data and logout. Django admin also checks staff permissions.
How are uploaded photos stored?	The database stores a file path in an ImageField; the image file is stored separately under MEDIA_ROOT. Development URL serving is enabled only in DEBUG.
How is profit calculated?	Net sales (gross totals − refunds) minus COGS (each item’s unit cost × quantity). Margin is profit ÷ net sales. The same `build_analytics()` helper feeds the KPI cards, charts, and CSV export.
Why are the analytics charts live without polling?	The existing WebSocket is reused: `record_sale()` / `record_return()` broadcast a lightweight event after commit, `base.html` dispatches a window event, and the page refetches the JSON data endpoint.
Why are migrations important?	The model is Python’s schema declaration; migrations version and apply schema changes to the database without manually editing SQLite.
What would you improve next?	Examples: stronger stock adjustment semantics and audit trail, automated UI tests, production media storage, environment-based secrets, and pagination/monitoring for larger datasets.
How do you know a change didn’t break another feature?	Run the focused test, the full suite, Django checks, and manually verify the affected user workflow and role boundaries.

> **When surprised by a change request:** ask for the intended result and edge cases. Repeat your understanding, identify the owning layer, make a small change, and demonstrate the test. It is acceptable to state an assumption before proceeding.

10. Caveats and production safety

- `settings.py` currently has `DEBUG = True` and a hardcoded development secret. These are not production settings. Use environment variables, set DEBUG false, and configure allowed hosts before deployment.
- Uploaded media is served by Django’s development helper only while DEBUG is enabled. A production deployment needs durable media storage and web-server/CDN delivery.
- Local demo passwords are intentionally simple and must never be reused in production.
- The README and project map may describe older behavior. Current source is authoritative: this version includes Stock Viewer, live stock, product image upload, day-count sales reports, returns and refunds, and the live analytics dashboard.
- SQLite is suitable for this capstone/local deployment. A multi-user production environment should evaluate a managed database and backup/restore procedures.

One-minute mental model

> **Routes choose views.** Views enforce access and coordinate forms, queries, and responses.
>
> **Models own data rules.** Migrations carry schema edits to the database.
>
> **Templates render screens.** JavaScript enhances the POS, but the server validates important business rules.
>
> **Tests prove behavior.** Follow one workflow from URL to database and back when debugging.

Study source: `docs/capstone-study-guide.html`. Generated PDF: `docs/AlvanzStock-Capstone-Study-Guide.pdf`. Verify commands and code paths against the current repository if the project changes.